IFNOTUS PRODUCTION SERVER REMEDIATION & ISPCONFIG MIGRATION

You are now implementing the remediation plan from the completed **IFNOTUS Production-Readiness & Architecture Audit dated 2026-08-27**.

This is a live server.

The system currently hosts:

- IFNOTUS production platform
- 7 existing legacy hosting tenants
- VoteBridge
- QuizSnap
- ExamFlow
- csdttu applications
- serverlabsttu
- additional internal/product applications
- MySQL
- PostgreSQL
- Redis
- BIND
- Postfix
- Dovecot
- Roundcube
- nginx

The objective is to turn this VPS into a **proper IFNOTUS hosting node** using:

```
IFNOTUS UI
    ↓
FastAPI
    ↓
HostingProvider
    ↓
ISPConfig Provider
    ↓
ISPConfig
    ↓
Linux hosting infrastructure
```

while preserving all existing working sites.

CRITICAL OPERATING RULE

DO NOT perform the entire migration in one pass.

Work strictly phase by phase.

After every phase:

1. run verification
2. report what changed
3. report what remains unchanged
4. report every command executed
5. report every service restarted
6. report every file modified
7. report every backup created
8. return PASS / PARTIAL / FAIL
9. STOP

Do not start the next phase until explicitly instructed.

CURRENT VERIFIED STATE

Current server:

```
Ubuntu 24.04.4 LTS
12 CPU cores
47 GiB RAM
242 GB root disk
Public IP: 80.241.223.82
```

Current public web engine:

```
nginx
```

Current IFNOTUS API:

```
127.0.0.1:8010
```

Current tenants:

```
7
provider=legacy
```

Current hosting provider state:

```
legacy     = LIVE
ispconfig  = CODE ONLY
olspanel   = STOPPED / PARKED
```

ISPConfig is NOT installed yet.

NON-NEGOTIABLE ARCHITECTURE

IFNOTUS remains the product.

Customers must NOT use raw ISPConfig.

Customers continue using:

```
https://domain.tld/cpanel
```

which resolves securely to the IFNOTUS hosting panel.

Staff continue using:

```
https://cpanel.ifnotus.space
```

Webmail convenience remains:

```
https://domain.tld/mail
```

which redirects into the IFNOTUS-supported webmail system.

ISPConfig becomes infrastructure only.

FINAL RESPONSIBILITY SPLIT

IFNOTUS OWNS

```
Customer identity
Authentication
Roles
Billing
MoMo
Paystack
Orders
Subscriptions
Coupons
Plans
Renewals
Support
Student hostname rules
Reserved hostname rules
Customer UI
Staff UI
/cpanel routing
/mail convenience routing
HostingProvider abstraction
Provisioning orchestration
Provider reconciliation
Application hosting UX
Django/Flask/FastAPI/Node deployment UX
Business audit logs
Notifications
```

ISPConfig OWNS AFTER MIGRATION

```
Hosting clients
Websites
Web server configuration
PHP site configuration
FTP accounts
Tenant databases where supported
Mail domains
Mailboxes
DNS zones if ISPConfig becomes DNS authority
SSL certificates for migrated tenants
Hosting templates
Disk quotas
```

```
Traffic quotas
Hosting-level site isolation
```

OPERATING SYSTEM / IFNOTUS RUNTIME OWNS

```
systemd
cgroups v2
Python runtime
Node runtime
Gunicorn
Uvicorn
application services
advanced CPU/RAM enforcement
product applications
server monitoring
```

PHASE A — IMMEDIATE SERVER SECURITY STABILIZATION

DO THIS BEFORE INSTALLING ISPCONFIG.

Current audit found unsafe public firewall exposure.

Inspect current state first:

```
ufw status numbered
ss -lntup
systemctl --type=service --state=running
```

Verify actual bind addresses before changing firewall.

Close unnecessary public access

These must NOT remain open publicly unless verified necessary:

```
3306 MySQL
6379 Redis
7080 OpenLiteSpeed admin
```

```
4958 OLSPanel
19999 Netdata
```

OLSPanel/OpenLiteSpeed are currently stopped and should not require public firewall rules.

Do NOT close:

```
22
25
53
80
443
465
587
993
```

without first determining their production purpose.

Review:

```
21
110
143
995
40000:40100
40110:40210
```

and determine whether they are actually needed.

Do not blindly remove FTP passive ranges until FTP policy is resolved.

NETDATA

Netdata currently binds publicly.

Change it so it is accessible only through:

```
localhost
```

or another explicitly secured administration path.

Do not expose port 19999 publicly.

Verify:

```
ss -lntp | grep 19999
```

after remediation.

SSH HARDENING

Audit:

```
PermitRootLogin
PasswordAuthentication
PubkeyAuthentication
AllowUsers
AllowGroups
```

There are conflicting SSH configuration fragments.

Find the effective configuration using:

```
sshd -T
```

Target:

```
root SSH password login = disabled
SSH key authentication = enabled
normal administration preserved
SFTP tenant Match rules preserved
```

Do not lock the administrator out.

Before restarting SSH:

1. verify current working key access
2. validate configuration with `sshd -t`
3. keep the current SSH session alive

FAIL2BAN

Current fail2ban only meaningfully protects SSH.

Audit possible jails for:

```
sshd
postfix
dovecot
nginx
roundcube
```

Do not enable excessive or inappropriate bans.

PHASE A VERIFICATION

Return:

```
UFW SAFE: yes/no
MYSQL PUBLIC: yes/no
REDIS PUBLIC: yes/no
NETDATA PUBLIC: yes/no
OLS PORTS PUBLIC: yes/no
SSH ROOT PASSWORD LOGIN: yes/no
SSHD CONFIG VALID: yes/no
LIVE WEBSITES STILL WORKING: yes/no
MAIL STILL WORKING: yes/no
DNS STILL WORKING: yes/no
```

STOP.

PHASE B — CLEAN THE PROVIDER ARCHITECTURE IN CODE

Do not install ISPConfig yet.

Inspect:

```
backend/app/services/platform/provisioning.py
backend/app/services/platform/lifecycle.py

backend/app/services/hosting_provider/
backend/app/integrations/ispconfig/
backend/app/integrations/olspanel/
```

The main issue is:

```
HostingProvider exists
BUT
ProvisioningEngine still directly performs legacy provisioning.
```

Refactor toward:

```
ProvisioningEngine
    ↓
HostingProvider factory
    ↓
legacy | ispconfig
```

Do NOT remove legacy.

Existing environments must continue using:

```
provider=legacy
```

New ISPConfig test environments will later use:

```
provider=ispconfig
```

HOSTING PROVIDER INTERFACE

Create or standardize provider operations around stable internal methods such as:

```
create_account()
update_account()
suspend_account()
```

```
unsuspend_account()
delete_account()

create_site()
update_site()
delete_site()

create_database()
delete_database()

create_ftp_user()
delete_ftp_user()

create_mail_domain()
create_mailbox()
delete_mailbox()

create_dns_zone()
create_dns_record()
update_dns_record()
delete_dns_record()

issue_ssl()

get_usage()
get_quota()

create_cron()
delete_cron()

health_check()
```

Do not expose ISPConfig implementation details to business services.

PROVIDER CAPABILITY MODEL

Add a provider capability system.

Example:

```
websites
databases
ftp
sftp
```

```
mail
dns
ssl
cron
usage
quotas
python_runtime
node_runtime
```

The frontend should not assume every provider supports every operation.

PROVIDER RECONCILIATION

Design a reconciliation service.

Purpose:

```
IFNOTUS says ACTIVE
but ISPConfig says missing
```

must be detectable.

The reconciliation job should compare:

```
hosting account
website
provider client
provider site
status
quota
```

and report drift.

Do not automatically perform destructive corrections yet.

IDEMPOTENCY

Provisioning must become safe to retry.

If the same provisioning job executes twice, it must not create:

```
two clients
two websites
two databases
duplicate DNS zones
duplicate mail domains
```

Introduce deterministic provider references or idempotency markers.

PHASE B TESTS

At minimum add unit tests for:

```
legacy provider selected correctly
ispconfig provider selected correctly
existing legacy environment remains legacy
provider create called once
retry does not duplicate
unsupported capability handled cleanly
provider failure keeps environment non-ACTIVE
```

Run tests.

STOP.

PHASE C — BACKUP BEFORE ISPCONFIG INSTALLATION

Before installing anything:

Create a fresh timestamped backup.

Back up:

```
/srv/apps/ifnotus
/srv/apps/ifnotus-customers
/srv/apps/csdttu*
/srv/apps/quiz*
/srv/apps/votebridge
/srv/apps/serverlabsttu
```

```
/etc/nginx
/etc/bind
/etc/postfix
/etc/dovecot
/etc/ssh
/etc/letsencrypt
/etc/systemd
```

Database backups:

```
MySQL full dump
PostgreSQL full dump
```

Also back up:

```
crontabs
firewall state
enabled systemd units
installed package list
listening-port snapshot
```

Generate SHA256 checksums.

Verify archives are readable.

Do NOT proceed if backups fail validation.

STOP.

PHASE D — RESOLVE FTP/SFTP OVERLAP

Current server contains:

```
OpenSSH SFTP
vsftpd code/history
Pure-FTPd-MySQL currently installed/listening
```

We do not want three competing transfer models.

Preferred IFNOTUS policy:

```
DEFAULT:
SFTP
chrooted
no interactive shell
```

FTP/FTPS should exist only if required by legacy compatibility.

Determine which service ISPConfig expects for the chosen installation.

Do not keep redundant daemons running unnecessarily.

Do NOT remove Pure-FTPd until ISPConfig installation requirements are known.

Return recommended final transfer architecture.

STOP.

PHASE E — ISPCONFIG INSTALLATION PLAN

Do not run the installer blindly.

Before installation identify:

```
existing nginx
existing MySQL 8
existing PostgreSQL
existing BIND
existing Postfix
existing Dovecot
existing Roundcube
existing certificates
existing product applications
existing tenant sites
```

Prepare an ISPConfig install strategy that preserves them.

Preferred web server:

```
nginx
```

Do NOT switch the production server to Apache merely to satisfy defaults.

ISPCONFIG INSTALL

Install the current stable ISPConfig 3 release appropriate for Ubuntu 24.04.

Do not use stale installation instructions.

Before executing commands, verify current official ISPConfig documentation.

After installation:

```
ISPConfig panel should function
nginx should function
MySQL should function
PostgreSQL should function
BIND should function
Postfix should function
Dovecot should function
Roundcube should function
existing product sites should be recoverable
```

ISPCONFIG ADMIN ACCESS

ISPConfig admin access is infrastructure-only.

Do not publish it as customer cPanel.

Restrict management access where practical.

PHASE E STOP CONDITION

Do NOT migrate tenants yet.

STOP once ISPConfig itself is healthy.

PHASE F — ISPCONFIG REMOTE API

Create a dedicated remote API user specifically for IFNOTUS.

Use least privilege.

Store credentials only in backend environment configuration:

```
ISPCONFIG_BASE_URL
ISPCONFIG_REMOTE_USER
ISPCONFIG_REMOTE_PASSWORD
ISPCONFIG_SERVER_ID
ISPCONFIG_TEMPLATE_MAP
```

Never expose these values to:

```
Vue
browser
customer
localStorage
API responses
logs
Git
```

Verify:

```
GET provider health
create test client
read test client
delete test client
```

through the IFNOTUS adapter.

Do not use real customers yet.

STOP.

PHASE G — COMPLETE ISPCONFIG CLIENT COVERAGE

The audit found these gaps:

```
client update                partial
client delete                partial
website update/delete        partial
subdomain/alias              missing
database user flows          partial
FTP                          missing
mail                         missing
DNS                          missing
SSL                          placeholder
cron                         missing
usage/quota                  partial
SFTP/shell                   missing
```

Implement only the methods IFNOTUS currently needs.

Priority:

```
1. client
2. website
3. database
4. FTP/SFTP
5. SSL
6. usage/quota
7. mail
8. DNS
9. cron
```

Use typed schemas.

Normalize ISPConfig errors.

Never return raw provider errors to customers.

STOP.

PHASE H — CREATE FIRST ISPCONFIG TEST TENANT

Create a NON-PAYING internal test environment.

Example:

```
provider=ispconfig
plan=Student Basic Test
hostname=isp-test.ifnotus.space
```

Test:

```
create hosting account
create website
create document root
upload index.html
serve website
issue SSL
create database
connect database
create SFTP/FTP access
upload file
verify isolation
suspend
verify site inaccessible/disabled
unsuspend
verify site returns
delete test environment
```

Record all provider IDs.

No existing customer should be touched.

STOP.

PHASE I — TENANT ISOLATION

Test real separation.

Create two test tenants:

```
Tenant A
Tenant B
```

Attempt:

```
A read B files
A modify B files
A access B database
A use B credentials
A traverse filesystem
A follow symlink outside root
A upload zip traversal payload
A consume excessive storage
A consume excessive processes
```

All cross-tenant access must fail.

FILE MANAGER

Keep the IFNOTUS file manager UI.

It must operate only within an authoritative provider-owned document root.

Validate:

```
realpath
symlinks
../ traversal
zip-slip
tar traversal
ownership changes
chmod
extraction destination
copy/move destination
```

Do not allow absolute filesystem paths from the browser.

STOP.

PHASE J — REAL RESOURCE ENFORCEMENT

Current audit says:

```
Disk = application level
CPU = best effort
```

```
RAM = best effort
Bandwidth = mostly display
```

This is not sufficient.

Implement and verify real enforcement.

Disk

Use actual OS/filesystem quota or provider-supported quota.

Customer SFTP uploads must not bypass it.

CPU

Use appropriate:

```
cgroups v2
systemd slices
CPUQuota
```

where ISPConfig does not sufficiently enforce the package requirement.

RAM

Use:

```
MemoryMax
```

or equivalent cgroup enforcement for application runtimes.

Processes

Use:

```
TasksMax
RLIMIT_NPROC
```

where appropriate.

Do not advertise a limit as "enforced" until tested.

RESOURCE STATUS MODEL

Expose distinction:

```
ALLOCATED
REPORTED
ENFORCED
```

Example:

```
512 MB RAM
Status: Enforced
```

versus:

```
20 GB bandwidth
Status: Monitored
```

Customer UI must be truthful.

STOP.

PHASE K — DOMAIN ARCHITECTURE

Final standard:

```
example.com
www.example.com
```

Hosting panel convenience:

```
https://example.com/cpanel
```

must continue to redirect securely into IFNOTUS.

Do NOT use:

```
:2083
ISPConfig :8080
```

for customers.

CPANEL ROUTING

Target:

```
domain.tld/cpanel
    ↓
fixed IFNOTUS portal URL
    ↓
authentication
    ↓
resolve host
    ↓
verify ownership
    ↓
/hosting/{environment}
```

Do not trust `Host` alone.

The user must own the mapped environment.

Avoid open redirects.

Use a fixed trusted portal origin.

Prefer short-lived SSO state/token if needed.

STOP.

PHASE L — DNS

Current state:

```
BIND authoritative
ns1.ifnotus.space
ns2.ifnotus.space
```

but both nameservers resolve to the same IP.

That is not resilient DNS.

Single writer rule

Choose exactly one authority for customer DNS.

Preferred after migration:

```
IFNOTUS UI
    ↓
HostingProvider
    ↓
ISPConfig DNS
    ↓
BIND
```

Do not keep:

```
IFNOTUS DB
+
manual BIND
+
ISPConfig
```

all writing independently.

DNS PROVIDER MODE

Continue supporting two customer scenarios.

Managed DNS

Customer points NS to IFNOTUS.

IFNOTUS controls records.

External DNS

Customer uses Cloudflare/registrar DNS.

IFNOTUS shows required records but does not mutate external DNS without integration.

DNS REDUNDANCY

Plan:

```
ns1.ifnotus.space → primary
ns2.ifnotus.space → separate server / secondary DNS provider
```

They should not permanently live on the same failure domain.

STOP.

PHASE M — MAIL ARCHITECTURE

Current mail:

```
Postfix
Dovecot
Roundcube
mail.ifnotus.space
```

Final user experience:

```
domain.tld/mail
    ↓
webmail
```

and, where the package includes mail:

```
mail.domain.tld
```

for SMTP/IMAP.

Mail should be plan-gated.

Student packages do not automatically need mail.

MAIL DNS

For hosted mail configure and verify:

```
MX
SPF
DKIM
DMARC
PTR/rDNS
```

Optional:

```
autodiscover
autoconfig
```

Do not sell serious business mail until reputation and delivery are tested against:

```
Gmail
Outlook
Yahoo
```

STOP.

PHASE N — SSL OWNERSHIP

Current tenant certificates are owned by direct Certbot automation.

Migrated tenant certificates should eventually be owned by ISPConfig.

Rule:

```
ONE CERTIFICATE
ONE OWNER
```

During migration:

```
legacy tenant → IFNOTUS Certbot
ispconfig tenant → ISPConfig Let's Encrypt
```

Do not allow two renewal systems to manage the same certificate.

STOP.

PHASE O — PRODUCT APPLICATION SEPARATION

These applications are NOT tenant hosting:

```
VoteBridge
QuizSnap
ExamFlow
csdttu
serverlabsttu
other platform products
```

Give them explicit classification:

```
resource_class=product
```

or equivalent.

They must never inherit:

```
tenant quotas
customer permissions
customer file-manager access
customer suspension actions
```

Super Admin UI should display separate areas:

```
Platform
Products
Tenants
Infrastructure
```

STOP.

PHASE P — ROLE HARDENING

Current roles:

```
superadmin
admin
operator
viewer
customer_care
customer
```

Clean permissions.

SUPER ADMIN

May:

```
manage servers
providers
hosting templates
staff
terminations
infrastructure
provider credentials
```

Destructive actions require confirmation.

ADMIN

May:

```
customers
billing
```

```
plans
domains
hosting operations
suspend/reactivate
```

No root shell.

OPERATOR

May:

```
diagnostics
files
DNS
mail
database operations
hosting operations
```

but reduce unnecessary host-wide write capability.

CUSTOMER CARE

Support/payment workflows only.

VIEWER

Read only.

CUSTOMER

Own resources only.

REMOVE STAFF PORTAL CROSSOVER

Customer portal routes should not silently become staff administration routes.

Prefer:

```
/customer APIs
```

for customer ownership

and:

```
/platform APIs
```

for staff.

Explicitly test IDOR and tenant boundary rules.

STOP.

PHASE Q — ADMIN 2FA

Require 2FA for:

```
superadmin
admin
operator
```

at minimum where their permissions allow sensitive operations.

Do not rely only on passwords for infrastructure staff.

STOP.

PHASE R — BACKUPS

Same-VPS backup is not disaster recovery.

Keep local fast backups.

Add encrypted offsite backup.

Back up:

```
tenant files
product files
IFNOTUS DB
MySQL
PostgreSQL
```

```
mail
DNS
ISPConfig DB/config
nginx
critical /etc config
```

Use a proven system such as:

```
restic
```

to remote object storage or backup server.

RESTORE TESTING

A backup is considered valid only after restore testing.

Create documented restore drills.

At least verify restoration of:

```
one tenant website
one tenant database
one mailbox
IFNOTUS DB
ISPConfig configuration
```

STOP.

PHASE S — MONITORING

Netdata may remain for initial host monitoring but must be private.

Monitor:

```
CPU
RAM
load
disk
inodes
```

```
network
nginx
PHP-FPM
MySQL
PostgreSQL
Redis
Postfix
Dovecot
BIND
SSL expiry
backup failures
ISPConfig API
IFNOTUS API
tenant resource usage
```

Create alert thresholds.

Later evaluate:

```
Prometheus
Grafana
Alertmanager
```

for multi-node infrastructure.

STOP.

PHASE T — FIRST LEGACY TENANT MIGRATION

Do not bulk migrate.

Pick one low-risk tenant.

Procedure:

```
1. create fresh backup
2. create ISPConfig client
3. create ISPConfig site
4. rsync files
5. preserve permissions appropriately
6. migrate/attach database
7. configure DNS
```

```
8. issue SSL
9. configure FTP/SFTP
10. smoke test
11. switch provider=ispconfig
12. monitor
```

Do NOT delete old files.

Keep legacy environment archived for at least:

```
7–14 days
```

or longer where appropriate.

STOP.

PHASE U — CUSTOMER PANEL PROVIDER WIRING

For environments where:

```
provider=legacy
```

existing services continue temporarily.

For:

```
provider=ispconfig
```

customer panel actions must use HostingProvider.

Wire:

```
domains
databases
email
FTP/SFTP
SSL
usage
cron
```

```
suspend
reactivate
```

Do not scatter ISPConfig HTTP calls across routers.

Everything goes through:

```
HostingProvider
```

STOP.

PHASE V — MODERN APPLICATION HOSTING

Do NOT force ISPConfig to become Render/Railway.

Build IFNOTUS application runtime separately.

Support:

```
Python
├── Django
├── Flask
└── FastAPI

Node
├── Express
├── Nest
├── Next.js
└── SvelteKit

Static
├── React
├── Vue
└── Svelte

PHP
├── Laravel
├── WordPress
└── generic PHP
```

PYTHON RUNTIME

Per app:

```
isolated virtualenv
runtime version
requirements
environment variables
Gunicorn/Uvicorn
systemd service
logs
health checks
reverse proxy
```

NODE RUNTIME

Per app:

```
isolated app directory
Node version
dependencies
environment variables
start command
systemd service
logs
health check
reverse proxy
```

Static SPA builds should not require permanent Node processes.

PHASE W — REMOVE OBSOLETE LEGACY CODE

Only after ISPConfig is stable and tenants migrated.

Candidates:

```
OLSPanel integration
OpenLiteSpeed leftovers
direct tenant nginx generation
```

```
direct Certbot for migrated tenants
new-tenant unix_identity creation
duplicate FTP daemon model
obsolete cpanel.* customer-vhost logic
```

Do NOT remove:

```
Billing
HostingProvider
DNS UX
reserved names
customer panel
application engine
business logic
```

Archive superseded documentation instead of losing migration history.

STOP.

PHASE X — DEFAULT PROVIDER CUTOVER

Only after:

```
multiple ISPConfig tenants tested
provisioning retries tested
suspend/reactivate tested
backups working
resource enforcement proven
customer panel works
ISPConfig reconciliation works
```

change:

```
HOSTING_PROVIDER_DEFAULT=legacy
```

to:

```
HOSTING_PROVIDER_DEFAULT=ispconfig
```

This affects NEW hosting sales only.

Existing legacy tenants remain legacy until migrated.

STOP.

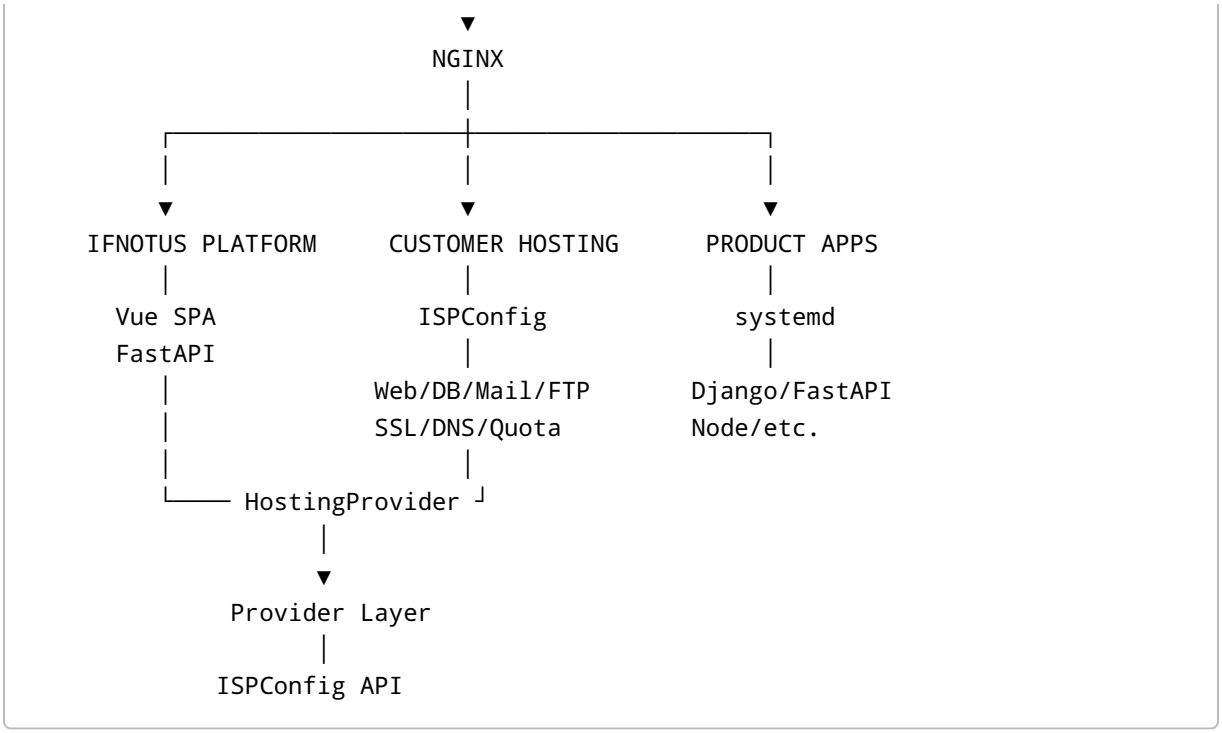
PHASE Y — PRODUCTION GATES

Do NOT declare IFNOTUS production-ready until all are PASS:

```
[ ] Firewall hardened
[ ] SSH hardened
[ ] Netdata private
[ ] OLS ports removed
[ ] ISPConfig installed
[ ] Remote API secured
[ ] Provider abstraction live
[ ] Provider reconciliation live
[ ] Idempotent provisioning
[ ] Test tenant works
[ ] Tenant isolation penetration tests pass
[ ] Disk quota real
[ ] CPU controls real where advertised
[ ] RAM controls real where advertised
[ ] Offsite backups verified
[ ] Restore test passed
[ ] DNS single writer
[ ] Secondary DNS
[ ] SSL single owner
[ ] Mail SPF/DKIM/DMARC/PTR tested
[ ] Staff 2FA
[ ] Cross-tenant API tests
[ ] Product apps separated operationally
[ ] Monitoring alerts functioning
```

FINAL TARGET SERVER ARCHITECTURE

```
                        INTERNET
                           │
                           ▼
                    AUTHORITATIVE DNS
                ns1 + independent ns2
                           │
```



Customers see only IFNOTUS.

FINAL CUSTOMER DOMAIN EXPERIENCE

Custom domain:

```
https://example.com
https://www.example.com

https://example.com/cpanel
    → IFNOTUS hosting panel

https://example.com/mail
    → webmail

mail.example.com
    → SMTP/IMAP where mail plan enabled
```

Student hosting:

```
https://surname.ifnotus.space
```

```
https://surname.ifnotus.space/cpanel
    → IFNOTUS hosting panel
```

Do not generate unnecessary mail subdomains for packages without email.

REQUIRED OUTPUT AFTER EVERY PHASE

Return:

```
{
  "phase": "",
  "status": "PASS | PARTIAL | FAIL",
  "changes_made": [],
  "files_modified": [],
  "services_restarted": [],
  "commands_executed": [],
  "backups_created": [],
  "tests_run": [],
  "tests_passed": [],
  "tests_failed": [],
  "security_findings": [],
  "downtime_observed": false,
  "existing_sites_verified": [],
  "rollback_required": false,
  "rollback_notes": "",
  "next_recommended_phase": ""
}
```

Also include a concise human-readable summary.

STARTING INSTRUCTION

Begin with **PHASE A ONLY: Immediate Server Security Stabilization**.

Do not install ISPConfig yet.

Do not modify the hosting provider default.

Do not migrate customers.

Do not remove OLSPanel packages yet.

Do not touch working nginx virtual hosts.

Inspect first, make only justified security changes, verify all existing public websites/services afterward, return the required report, then STOP.